Ex.no:5	DATA VISUALIZATION USING MATPLOTLIB – SALES DATA

Aim:

To visualize air quality features and their distributions using line charts, bar charts, pie charts, histograms, scatter plots, box plots, and multiple line charts using Matplotlib.

Algorithm:

Objective:

To visualize the PM2.5 pollution trend across the available records.

Steps:

1. Load the air quality dataset.
2. Select the City and PM2.5 columns.
3. Plot PM2.5 values using a line chart.
4. Add title and axis labels.
5. Display the graph.

Interpretation:

The line chart shows the variation in PM2.5 concentration across different records and helps identify changes in air pollution levels.

Program 1: Line Chart

```
plt.plot(df["PM2.5"].head(20))
plt.title("PM2.5 Pollution Trend")
plt.xlabel("Record Number")
plt.ylabel("PM2.5")
plt.show()
```

ALGORITHM 2: BAR CHART

Objective:

To compare average PM2.5 levels across different cities.

Steps:

1. Group the dataset based on City.
2. Calculate the average PM2.5 for each city.
3. Create a bar chart using the city-wise average values.
4. Add title and axis labels.
5. Display the chart.

Interpretation:

The bar chart helps compare the average PM2.5 pollution levels among different cities.

Program 2: Bar Chart

```
city_pm25 = df.groupby("City")["PM2.5"].mean()
plt.figure(figsize=(10,5))
plt.bar(city_pm25.index, city_pm25.values)
plt.title("Average PM2.5 by City")
plt.xlabel("City")
plt.ylabel("Average PM2.5")
plt.xticks(rotation=45)
plt.show()
```

ALGORITHM 3: PIE CHART

Objective:

To visualize the distribution of average PM2.5 levels across cities.

Steps:

1. Group the dataset based on City.
2. Calculate the average PM2.5 for each city.
3. Create a pie chart using the average PM2.5 values.
4. Display percentage labels.
5. Add a suitable title.
6. Display the chart.

Interpretation:

The pie chart shows the percentage contribution of each city to the total average PM2.5 concentration.

Program 3: Pie Chart

```
city_pm25 = df.groupby("City")["PM2.5"].mean()
```

```
plt.figure(figsize=(8,8))
plt.pie(city_pm25.values, labels=city_pm25.index, autopct="%1.1f%%")
plt.title("PM2.5 Distribution by City")
plt.show()
```

ALGORITHM 4: HISTOGRAM

Objective:

To visualize the distribution of PM2.5 pollution values.

Steps:

1. Select the PM2.5 column.
2. Remove missing values.
3. Divide the PM2.5 values into bins.
4. Create a histogram.
5. Add title and axis labels.
6. Display the histogram.

Interpretation:

The histogram shows the frequency distribution of PM2.5 values and indicates how frequently different pollution levels occur.

Program 4: Histogram

```
plt.figure(figsize=(8,5))
plt.hist(df["PM2.5"].dropna(), bins=10)
plt.title("PM2.5 Distribution")
plt.xlabel("PM2.5")
plt.ylabel("Frequency")
plt.show()
```

ALGORITHM 5: SCATTER PLOT

Objective:

To visualize the relationship between PM2.5 and PM10 pollution levels.

Steps:

1. Select PM2.5 as the X-axis variable.
2. Select PM10 as the Y-axis variable.
3. Remove missing values.

4. Create a scatter plot.
5. Add title and axis labels.
6. Display the graph.

Interpretation:

The scatter plot shows the relationship between PM2.5 and PM10 concentrations. It helps identify whether the two pollutants vary together.

Program 5: Scatter Plot

```
plot_data = df[["PM2.5", "PM10"]].dropna()
plt.figure(figsize=(8,5))
plt.scatter(plot_data["PM2.5"], plot_data["PM10"])
plt.title("PM2.5 vs PM10")
plt.xlabel("PM2.5")
plt.ylabel("PM10")
plt.show()
```

ALGORITHM 6: BOX PLOT

Objective:

To visualize the distribution of PM2.5 pollution and identify possible outliers.

Steps:

1. Select the PM2.5 column.
2. Remove missing values.
3. Create a box plot.
4. Add a suitable title and axis label.
5. Display the plot.

Interpretation:

The box plot shows the distribution, median, spread, and possible outliers of PM2.5 pollution values.

Program 6: Box Plot

```
plt.figure(figsize=(6,5))
plt.boxplot(df["PM2.5"].dropna())
plt.title("PM2.5 Box Plot")
plt.ylabel("PM2.5")
plt.show()
```

ALGORITHM 7: MULTIPLE LINE CHARTS

Objective:

To compare PM2.5 and PM10 pollution levels across different records.

Steps:

1. Select the PM2.5 and PM10 values.
2. Plot PM2.5 values using a line chart.
3. Plot PM10 values using another line chart.
4. Add title, axis labels and legend.
5. Display the graph.

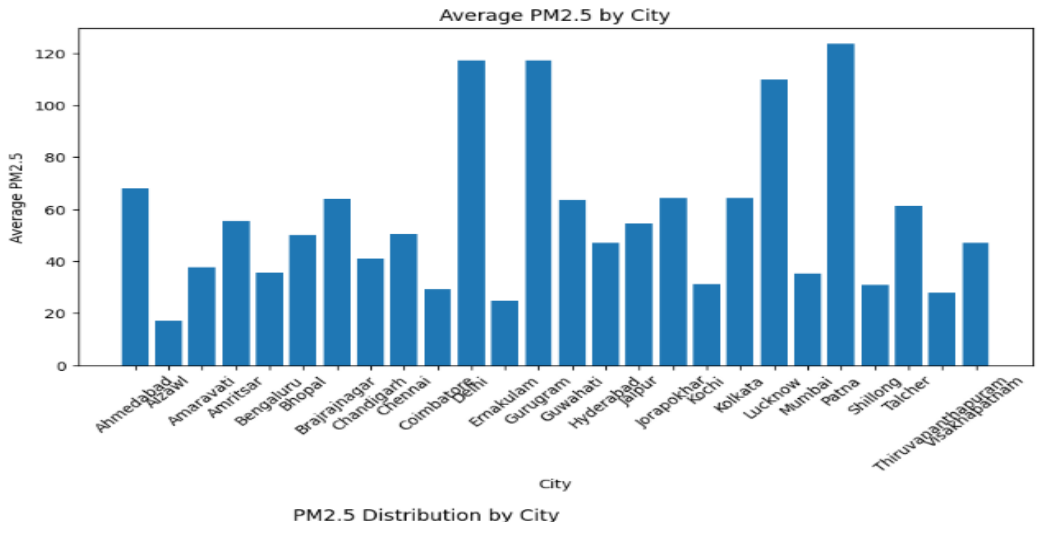
Interpretation:

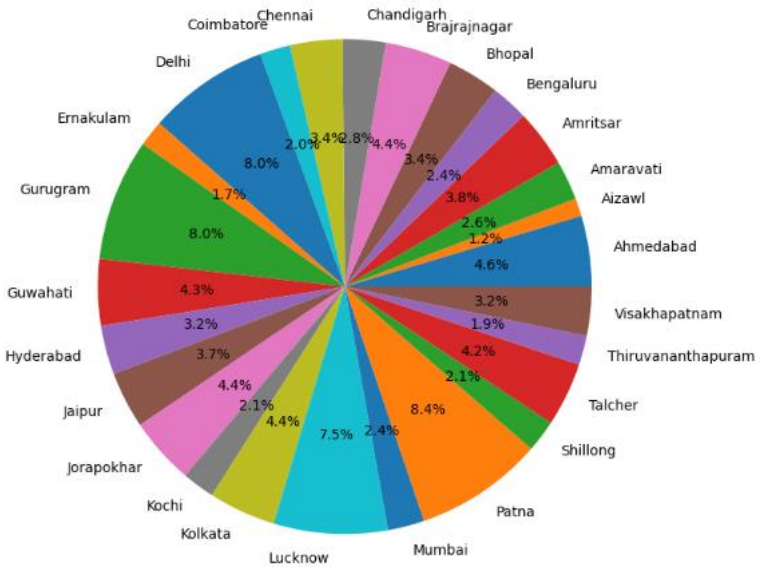
The multiple line chart compares PM2.5 and PM10 pollution levels and shows how both pollutants vary across different records.

Program 7: Multiple Line Charts

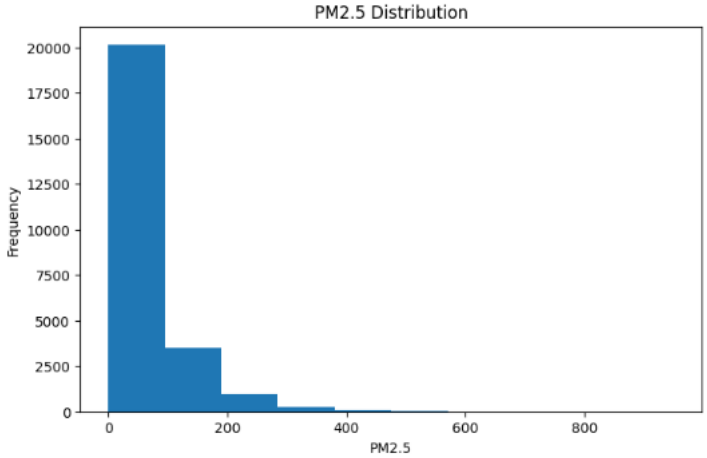
```
data = df[["PM2.5", "PM10"]].head(20)

plt.figure(figsize=(10,5))

plt.plot(data["PM2.5"].values, label="PM2.5")

plt.plot(data["PM10"].values, label="PM10")

plt.title("PM2.5 and PM10 Pollution")

plt.xlabel("Record Number")

plt.ylabel("Pollution Level")

plt.legend()

plt.show()
```

Output:

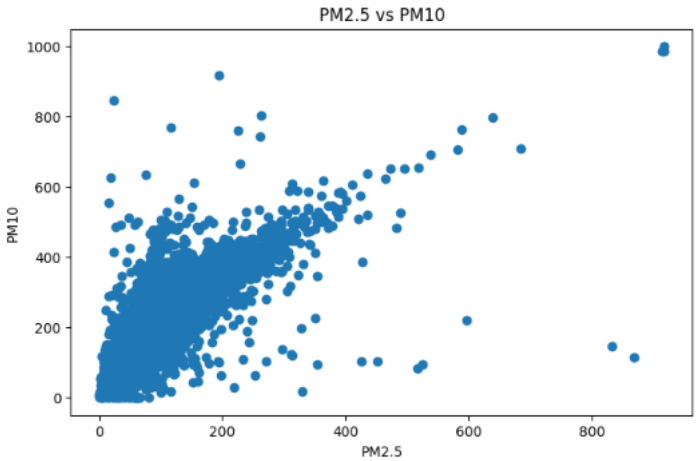


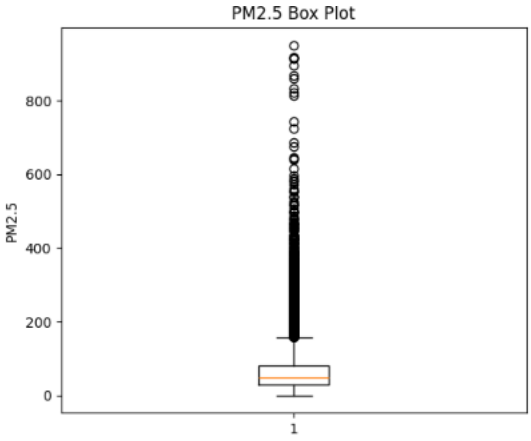


PM2.5 Distribution



PM2.5 vs PM10





Result: